

Complete Terminal CLI Migration Guide: Single Vultr Ubuntu + Apache + Postgres

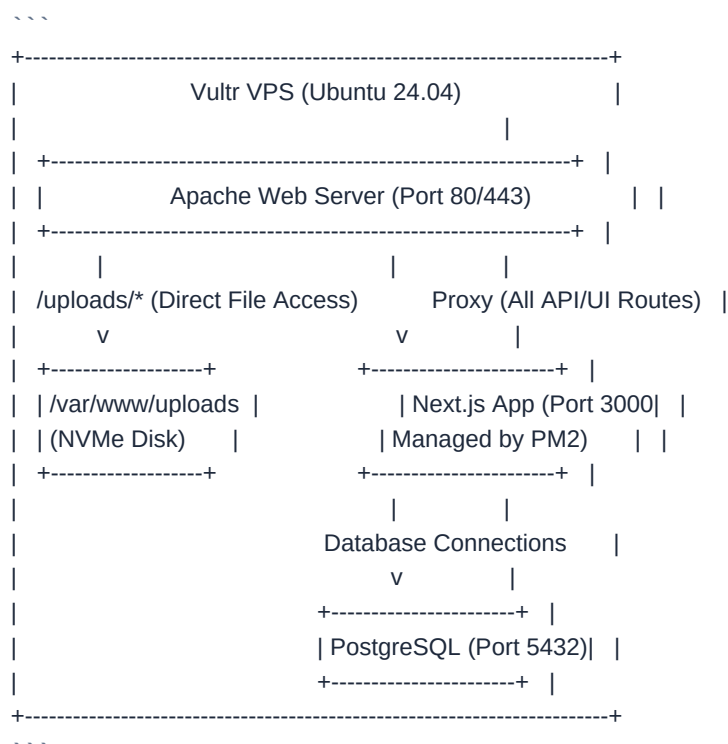
```

**Target Environment:** Single Vultr VPS (2 vCPU, 2GB RAM, 80GB NVMe)
**Operating System:** Ubuntu 24.04 LTS (CLI / Headless)
**Web Server:** Apache2 (`mod_proxy` + `mod_proxy_http`)
**Database:** PostgreSQL 16 (Local Daemon)
**Process Manager:** PM2
**SSL Certificate:** Let's Encrypt Certbot (Free HTTPS)

```

1. [Prerequisites & System Architecture](#1-prerequisites--system-architecture)
2. [Step 1: Server Provisioning & Swap Memory Setup](#step-1-server-provisioning--swap-memory-setup)
3. [Step 2: Installing Dependencies (Node, Postgres, Apache, PM2)](#step-2-installing-dependencies-node-postgres-apache-pm2)
4. [Step 3: PostgreSQL Database Setup](#step-3-postgresql-database-setup)
5. [Step 4: Codebase Modifications for Self-Hosting](#step-4-codebase-modifications-for-self-hosting)
6. [Step 5: Codebase Transfer, Configuration & Building](#step-5-codebase-transfer-configuration--building)
7. [Step 6: Process Management with PM2](#step-6-process-management-with-pm2)
8. [Step 7: Apache Reverse Proxy & SSE Stream Configuration](#step-7-apache-reverse-proxy--sse-stream-configuration)
9. [Step 8: SSL Certificate Setup (Certbot)](#step-8-ssl-certificate-setup-certbot)
10. [Step 9: Updating Client Apps (Android APKs & Tablets)](#step-9-updating-client-apps-android-apks--tablets)
11. [Step 10: Daily Automated Backup Script](#step-10-daily-automated-backup-script)
12. [Verification & Troubleshooting Checklist](#verification--troubleshooting-checklist)

Component Topology on Single VPS



Step 1: Server Provisioning & Swap Memory Setup

1.1 Create Vultr Server Instance

- **Cloud Provider:** Vultr Cloud Compute
- **OS:** Ubuntu 24.04 LTS x64
- **Plan:** Regular Cloud Compute or High Performance NVMe (2 vCPU, 2GB RAM, 80GB NVMe)
- **Location:** Select closest region to Belize / Central America (e.g. Miami, FL)

1.2 SSH Connection

Open your local terminal and connect as `root`:

```
```bash
ssh root@YOUR_VULTR_SERVER_IP
```
```

1.3 Create a 2GB NVMe Swap File (Critical for 2GB RAM)

To prevent Out-Of-Memory (OOM) crashes during `npm run build` or traffic spikes:

```
```bash
```

## Allocate 2GB swap file

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
```

## Make permanent across reboots

```
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

## Verify swap is active

```
free -h
```
```

Step 2: Installing Dependencies (Node, Postgres, Apache, PM2)

Run the following commands in sequence:

```
```bash
```

### 1. Update system packages

```
sudo apt update && sudo apt upgrade -y
```

### 2. Install Node.js 20.x LTS

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs build-essential git
```

### 3. Verify Node and NPM versions

```
node -v # Should be v20.x.x
npm -v # Should be 10.x.x
```

### 4. Install Apache2 & Utilities

```
sudo apt install -y apache2 certbot python3-certbot-apache
```

### 5. Install PostgreSQL 16

```
sudo apt install -y postgresql postgresql-contrib
```

## 6. Install PM2 globally

```
sudo npm install -g pm2
` ``
```

---

### Step 3: PostgreSQL Database Setup

#### 3.1 Initialize Database and User

```
` `` bash
```

#### Switch to postgres user and open psql prompt

```
sudo -u postgres psql
` ``
```

Execute the SQL queries inside the `psql` interface:

```
` `` sql
CREATE DATABASE signage_db;
CREATE USER signage_user WITH PASSWORD 'TropicAirSignage2026!';
GRANT ALL PRIVILEGES ON DATABASE signage_db TO signage_user;
ALTER DATABASE signage_db OWNER TO signage_user;
\q
` ``
```

---

### Step 4: Codebase Modifications for Self-Hosting

Before uploading your code to Vultr, make these **\*\*2 small edits\*\*** in your project repository so it runs without Vercel:

#### 4.1 Update `next.config.ts` for Standalone Output

Open `next.config.ts` and set `output: 'standalone'`:

```
` `` typescript
import type { NextConfig } from "next";
import withPWAInit from "@ducanh2912/next-pwa";

const withPWA = withPWAInit({
 dest: "public",
 disable: process.env.NODE_ENV === "development",
 register: true,
 cacheOnFrontEndNav: true,
});

const nextConfig: NextConfig = {
 output: 'standalone', // <-- ADD THIS LINE
 turbopack: {},
 // ... rest of headers and config
};

export default withPWA(nextConfig);
` ``
```

#### 4.2 Update Media Upload Handler (`src/app/api/media/route.ts`)

Replace `@vercel/blob` with local NVMe file storage writes:

```
` `` typescript
import { NextResponse } from 'next/server';
import { prisma } from '@lib/prisma';
import { getSession } from 'next-auth';
import { authOptions } from '@lib/auth';
```

```

import { logAudit } from '@lib/audit';
import { writeFile, mkdir } from 'fs/promises';
import path from 'path';

export async function POST(request: Request) {
 try {
 const session = await getServerSession(authOptions);
 const contentType = request.headers.get('content-type') || '';

 // ... (Keep existing JSON embed handler) ...

 // Handle FormData File Uploads to Local Disk
 const data = await request.formData();
 const file: File | null = data.get('file') as unknown as File;

 if (!file) {
 return NextResponse.json({ success: false, error: 'No file provided' }, { status: 400 });
 }

 // Save directly to local /public/uploads directory
 const uploadDir = path.join(process.cwd(), 'public', 'uploads');
 await mkdir(uploadDir, { recursive: true });

 const uniqueFilename = `${Date.now()}-${file.name.replace(/^[a-zA-Z0-9.-_]/g, '')}`;
 const filePath = path.join(uploadDir, uniqueFilename);

 const bytes = await file.arrayBuffer();
 await writeFile(filePath, Buffer.from(bytes));

 const fileUrl = `/uploads/${uniqueFilename}`;
 const folderId = data.get('folderId') as string | null;
 const type = file.type.startsWith('video/') ? 'video' : 'image';

 const mediaAsset = await prisma.mediaAsset.create({
 data: {
 name: file.name,
 url: fileUrl,
 type: type,
 folderId: folderId || null,
 }
 });

 await logAudit('UPLOAD_MEDIA', `Uploaded media file ${file.name}`, session);
 return NextResponse.json({ success: true, media: mediaAsset });
 } catch (error) {
 console.error("Upload error:", error);
 return NextResponse.json({ success: false, error: 'Upload failed' }, { status: 500 });
 }
}

```

---

## Step 5: Codebase Transfer, Configuration & Building

### 5.1 Clone Codebase to Server

On your Vultr server CLI:

```
```bash
```

Create application directory

```

sudo mkdir -p /var/www/signage
sudo chown -R $USER:$USER /var/www/signage
cd /var/www/signage

```

Clone your GitHub repository

```
git clone https://github.com/YOUR_ORGANIZATION/signage.git .
```

Create uploads folder and grant web permissions

```
mkdir -p public/uploads
sudo chmod -R 775 public/uploads
```
```

### 5.2 Create Production Environment File (`.env`)

Create the `.env` file inside `/var/www/signage/.env`:

```
```bash
nano /var/www/signage/.env
```
```

Paste the following variables:

```
```env
DATABASE_URL="postgresql://signage_user:TropicAirSignage2026!@localhost:5432/signage_db?schema=public"
NEXTAUTH_URL="https://signage.tropicair.com"
NEXTAUTH_SECRET="PasteYourGeneratedRandomSecretHere"
NODE_ENV="production"
PORT=3000
```
```

\*(Generate secret using `openssl rand -base64 32`)\*.

### 5.3 Install Dependencies & Run Database Migrations

```
```bash
```

Install npm packages

```
npm install
```

Generate Prisma Client & Push Database Schema

```
npx prisma generate
npx prisma db push
```

Build production application

```
npm run build
```
```

---

## Step 6: Process Management with PM2

Keep Next.js running 24/7 in the background with automatic restart on reboot:

```
```bash
cd /var/www/signage
```

Start Next.js standalone process with PM2

```
pm2 start npm --name "signage-app" -- start
```

Verify process status

```
pm2 status
```

Save PM2 state for reboot persistence

```
pm2 save
```

Setup PM2 auto-launch service on OS boot

pm2 startup

(Copy and execute the command PM2 prints in terminal)

...

Step 7: Apache Reverse Proxy & SSE Stream Configuration

7.1 Enable Required Apache Modules

```
```bash
sudo a2enmod proxy proxy_http headers ssl rewrite
```
```

7.2 Create Apache VirtualHost File

Create `/etc/apache2/sites-available/signage.conf`:

```
```bash
sudo nano /etc/apache2/sites-available/signage.conf
```
```

Paste the following complete configuration:

```
```apache
<VirtualHost *:80>
 ServerName signage.tropicair.com
 # Replace with your server domain or IP address

 DocumentRoot /var/www/signage

 # 1. Direct NVMe Media File Access (Zero CPU overhead)
 Alias /uploads "/var/www/signage/public/uploads"
 <Directory "/var/www/signage/public/uploads">
 Options -Indexes +FollowSymLinks
 AllowOverride None
 Require all granted
 </Directory>

 # 2. Real-Time Server-Sent Events (SSE) Stream - Unbuffered Instant Sync
 <Location "/api/screens">
 ProxyPass "http://127.0.0.1:3000/api/screens/" flushpackets=on
 SetEnv no-gzip 1
 </Location>

 # 3. Proxy all UI & API traffic to Next.js on port 3000
 ProxyPreserveHost On
 ProxyPass / http://127.0.0.1:3000/
 ProxyPassReverse / http://127.0.0.1:3000/

 # Security Headers
 Header always set X-Content-Type-Options "nosniff"
 Header always set X-Frame-Options "SAMEORIGIN"
 Header always set X-XSS-Protection "1; mode=block"

 ErrorLog ${APACHE_LOG_DIR}/signage_error.log
 CustomLog ${APACHE_LOG_DIR}/signage_access.log combined
</VirtualHost>
```
```

7.3 Enable Site and Reload Apache

```
```bash
sudo a2dissite 000-default.conf
sudo a2ensite signage.conf
```

```
sudo apache2ctl configtest # Should return 'Syntax OK'
sudo systemctl reload apache2
```
```

Step 8: SSL Certificate Setup (Certbot)

Secure your server with free Let's Encrypt SSL:

```
```bash
```

### Point your DNS (signage.tropicair.com) to your Vultr IP before running this

```
sudo certbot --apache -d signage.tropicair.com
```
```

Select option `2` to automatically redirect all HTTP traffic to HTTPS. Certbot automatically configures auto-renewal!

Step 9: Updating Client Apps (Android APKs & Tablets)

Update the hardcoded URLs in your client source code before building final APKs:

9.1 Android TV App (`android-app/.../MainActivity.kt`)

Update line 39:

```
```kotlin
// Change from old Hostinger URL to:
webView.loadUrl("https://signage.tropicair.com/tv")
```
```

9.2 Tablet App (`VVTabletApp/capacitor.config.json`)

Update line 6:

```
```json
{
 "appId": "com.tropicair.vvtablet",
 "appName": "VVTabletApp",
 "webDir": "www",
 "server": {
 "url": "https://signage.tropicair.com/tablet",
 "cleartext": true
 }
}
```
```

Step 10: Daily Automated Backup Script

Create an automatic daily backup script to store database dumps and media on NVMe:

```
```bash
sudo nano /usr/local/bin/backup_signage.sh
```
```

Paste the following script:

```
```bash
#!/bin/bash
BACKUP_DIR="/var/backups/signage"
DATE=$(date +%Y-%m-%d_%H%M%S)
mkdir -p $BACKUP_DIR
```

## Dump PostgreSQL Database

PGPASSWORD="TropicAirSignage2026!" pg\_dump -u signage\_user -h localhost signage\_db > \$BACKUP\_DIR/db\_\$(date +%Y%m%d).sql

## Keep only last 14 days of backups

```
find $BACKUP_DIR -type f -mtime +14 -name "*.sql" -delete
```

Make executable and add to crontab:

```
```bash
sudo chmod +x /usr/local/bin/backup_signage.sh
(crontab -l 2>/dev/null; echo "0 3 * * * /usr/local/bin/backup_signage.sh") | crontab -
```
```

---

## Verification & Troubleshooting Checklist

| Test Item                  | Verification Command / Step               | Expected Result                                |
|----------------------------|-------------------------------------------|------------------------------------------------|
|                            | :---   :---   :---                        |                                                |
| **Next.js Process**        | `pm2 status`                              | Status: `online`, CPU: `< 1%`, RAM: `< 250MB`  |
| **Apache Status**          | `sudo systemctl status apache2`           | `active (running)`                             |
| **PostgreSQL Connection**  | `sudo -u postgres psql -d signage_db`     | Successfully connects                          |
| **Media Upload Test**      | Upload a 10MB video via CMS Media Library | Saves to `/var/www/signage/public/uploads`     |
| **SSE Real-Time Sync**     | Change playlist on CMS                    | TV updates in `< 1 second` without page reload |
| **Tablet Remote Override** | Press button on `/tablet` page            | TV displays override overlay immediately       |
| **Memory Buffer**          | `free -h`                                 | Swap active, `< 1GB` RAM free                  |

---

\*Guide compiled for Tropic Air Digital Signage Operations (100 Screens Infrastructure Migration).\*